after taking the action. The $V(S_{t+1})$ is calculated by just taking the maximum Q value of the subsequent state S_{t+1}. This is the bootstrapping term we learned about in the DQN chapter. We will include this term in the 2D Ising model later in this chapter.

If you run the training loop and plot the grid again, you should see something like this:

```
>>> fig,ax = plt.subplots(2,1)
>>> for i in range(size[0]):
        ax[0].scatter(np.arange(len(losses[i])),losses[i])
>>> print(grid,grid.sum())
>>> ax[1].imshow(np.expand_dims(grid,0))
```

The first plot in figure 9.17 is a scatter plot of the losses over each epoch for each agent (each color is a different agent). You can see that the losses all fall and plateau around 30 epochs. The bottom plot is our Ising model grid, of course, and you can see that it has organized into two domains that are all completely aligned with each other. The lighter part in the middle is a group of agents that are aligned in the up (`1`) direction, and the rest are aligned in the down (`0`) direction. This is much better than the random distribution we started off with, so our MARL algorithm definitely worked in solving this 1D Ising model.

Figure 9.17. Top: The losses for each agent over the training epochs. You can see that they all decrease and at a minimum at around 30 epochs or so. Bottom: The 1D Ising model after maximizing rewards (minimizing energy). You can see that all the electrons are clustered together into domains where they are all oriented the same way.

